



UNSW
SYDNEY

COMP4141

Theory of Computation

Lecture 6: Context-Free Languages

Administrivia

- Formatif Tasks 2 & 3 viewable tonight.
 - Task 2: Design DFAs/NFAs given in Automata Tutor
 - In your tutorial explain why one of them works to your tutor (this week or next week)
 - Task 3: Regular expressions and non-regularity
- Remaining Formatif tasks up on Friday.
- This week's tutorial: design an imperative programming language

Context-Free Languages

Regular languages have many wonderful properties, but not all languages are regular. (E.g. $\{ a^i b^i \mid i \in \mathbb{N} \}$, arithmetic expressions)

Next, we'll study a more powerful class of languages, the *context-free languages* (CFLs).

CFLs were identified in the 1950's by linguist Noam Chomsky, as a natural place in a hierarchy of languages, which included the regular languages.

Just as DFAs represent “computation with finite memory”, CFLs capture the idea of “computation with simple recursion”.

Outline

Context-Free Grammars

Parse Trees

Closure Properties of CFLs

Pushdown Automata

Equivalence of CFGs and PDAs

Grammars other than CFGs

End Matter

Formal Definition of Context-Free Grammars

Definition

A *context-free grammar* (CFG) is a 4-tuple (N, Σ, P, S) , where

- 1 N is a finite set of *variables*,
- 2 Σ is a finite set, disjoint from N , of *terminals*,
- 3 $P \subseteq N \times (N \cup \Sigma)^*$ is a finite set of *rules*, and
- 4 $S \in N$ is the *start variable*.

Variables are often called *non-terminal symbols*, terminals are often called *terminal symbols*, rules also go under the name *productions*, and the start variable is also known as the *sentence symbol*.

Notational Conventions for CFGs

Typically,

- upper case letters $A, B, S, \dots$ are used for variables,
- $a, b, c, 0, 1 \dots$ for terminals,
- w, x, y, z for strings of terminals (Σ^*), and
- $\alpha, \beta, \gamma, \dots$ for strings of terminals and/or variables $((N \cup \Sigma)^*)$.

Productions are written as in

$$A \rightarrow aBc$$

Here

- A is the left-hand side (LHS), also called the *head*, and
- aBc is the right-hand side (RHS), also called the *body*.

Several productions with common heads can be combined:

$$A \rightarrow a \mid Aa \mid bAb$$

Example

$$G = (\{S\}, \{0, 1\}, \{S \rightarrow \epsilon \mid 0S1\}, S)$$

Derivations

The language of a given CFG, $G = (N, \Sigma, P, S)$, can be characterized using the concept of a *derivation*.

Definition

Derivation step: $\alpha A \beta \Rightarrow_G \alpha \gamma \beta$ whenever $A \rightarrow \gamma \in P$.

Define $\Rightarrow_G^*$ to be the *reflexive transitive closure* of $\Rightarrow_G$. That is, $\alpha \Rightarrow_G^* \beta$ if we can get from α to β in zero or more steps.

The *language of G* is

$$L(G) = \{ w \in \Sigma^* \mid S \Rightarrow_G^* w \}$$

Example

Example

If

$$G = (\{S\}, \{0, 1\}, \{S \rightarrow \epsilon \mid 0S1\}, S)$$

then

$S \Rightarrow_G^* 0011$ because

S

Example

Example

If

$$G = (\{S\}, \{0, 1\}, \{S \rightarrow \epsilon \mid 0S1\}, S)$$

then

$S \Rightarrow_G^* 0011$ because

$$S \Rightarrow_G 0S1$$

Example

Example

If

$$G = (\{S\}, \{0, 1\}, \{S \rightarrow \epsilon \mid 0S1\}, S)$$

then

$S \Rightarrow_G^* 0011$ because

$$S \Rightarrow_G 0S1 \Rightarrow_G 00S11$$

Example

Example

If

$$G = (\{S\}, \{0, 1\}, \{S \rightarrow \epsilon \mid 0S1\}, S)$$

then

$S \Rightarrow_G^* 0011$ because

$$S \Rightarrow_G 0S1 \Rightarrow_G 00S11 \Rightarrow_G 0011.$$

Example

Example

If

$$G = (\{S\}, \{0, 1\}, \{S \rightarrow \epsilon \mid 0S1\}, S)$$

then

$S \Rightarrow_G^* 0011$ because

$$S \Rightarrow_G 0S1 \Rightarrow_G 00S11 \Rightarrow_G 0011.$$

Exercise: Show $L(G) = \{ 0^i 1^i \mid i \in \mathbb{N} \}$

Example: Grammar for Balanced Parentheses

Example

Consider

$$G = (\{S\}, \{ (,) \}, \{ S \rightarrow \epsilon \mid (S) \mid SS \}, S)$$

then

$S \Rightarrow_G^* ()(())$ because

S

Example: Grammar for Balanced Parentheses

Example

Consider

$$G = (\{S\}, \{ (,) \}, \{ S \rightarrow \epsilon \mid (S) \mid SS \}, S)$$

then

$S \Rightarrow_G^* ()(())$ because

$$S \Rightarrow_G SS$$

Example: Grammar for Balanced Parentheses

Example

Consider

$$G = (\{S\}, \{ (,) \}, \{ S \rightarrow \epsilon \mid (S) \mid SS \}, S)$$

then

$S \Rightarrow_G^* ()(())$ because

$$S \Rightarrow_G SS \Rightarrow_G (S)S$$

Example: Grammar for Balanced Parentheses

Example

Consider

$$G = (\{S\}, \{ (,) \}, \{ S \rightarrow \epsilon \mid (S) \mid SS \}, S)$$

then

$S \Rightarrow_G^* ()(())$ because

$$S \Rightarrow_G SS \Rightarrow_G (S)S \Rightarrow_G ()S$$

Example: Grammar for Balanced Parentheses

Example

Consider

$$G = (\{S\}, \{ (,) \}, \{ S \rightarrow \epsilon \mid (S) \mid SS \}, S)$$

then

$S \Rightarrow_G^* ()(())$ because

$$S \Rightarrow_G SS \Rightarrow_G (S)S \Rightarrow_G ()S \Rightarrow_G ()(S)$$

Example: Grammar for Balanced Parentheses

Example

Consider

$$G = (\{S\}, \{ (,) \}, \{ S \rightarrow \epsilon \mid (S) \mid SS \}, S)$$

then

$S \Rightarrow_G^* ()()$ because

$$S \Rightarrow_G SS \Rightarrow_G (S)S \Rightarrow_G ()S \Rightarrow_G ()(S) \Rightarrow_G ()((S))$$

Example: Grammar for Balanced Parentheses

Example

Consider

$$G = (\{S\}, \{ (,) \}, \{ S \rightarrow \epsilon \mid (S) \mid SS \}, S)$$

then

$S \Rightarrow_G^* ()()$ because

$$S \Rightarrow_G SS \Rightarrow_G (S)S \Rightarrow_G ()S \Rightarrow_G ()(S) \Rightarrow_G ()((S)) \Rightarrow_G ()().$$

Example: Grammar for Palindromes

Example

Let $\Sigma = \{0, 1\}$ and consider $G = (\{S, T\}, \Sigma, R, S)$ with rules R given by:

$$\begin{aligned} S &\rightarrow 0S0 \mid 1S1 \mid T \\ T &\rightarrow \epsilon \mid 0 \mid 1 \end{aligned}$$

Exercise: Show that $L(G) = \{w \in \Sigma^* : w = \text{rev}(w)\}$

Example: Small English Language

Example

$\langle \text{SENTENCE} \rangle$	$\rightarrow$	$\langle \text{NOUN-PHRASE} \rangle \langle \text{VERB-PHRASE} \rangle$
$\langle \text{NOUN-PHRASE} \rangle$	$\rightarrow$	$\langle \text{CX-NOUN} \rangle \mid \langle \text{CX-NOUN} \rangle \langle \text{PREP-PHRASE} \rangle$
$\langle \text{VERB-PHRASE} \rangle$	$\rightarrow$	$\langle \text{CX-VERB} \rangle \mid \langle \text{CX-VERB} \rangle \langle \text{PREP-PHRASE} \rangle$
$\langle \text{PREP-PHRASE} \rangle$	$\rightarrow$	$\langle \text{PREP} \rangle \langle \text{CX-NOUN} \rangle$
$\langle \text{CX-NOUN} \rangle$	$\rightarrow$	$\langle \text{ARTICLE} \rangle \langle \text{NOUN} \rangle$
$\langle \text{CX-VERB} \rangle$	$\rightarrow$	$\langle \text{VERB} \rangle \mid \langle \text{VERB} \rangle \langle \text{NOUN-PHRASE} \rangle$
$\langle \text{ARTICLE} \rangle$	$\rightarrow$	a the
$\langle \text{NOUN} \rangle$	$\rightarrow$	boy girl flower
$\langle \text{VERB} \rangle$	$\rightarrow$	touches like see
$\langle \text{PREP} \rangle$	$\rightarrow$	with

10 variables, 9 terminals and 18 rules.

$\langle \text{SENTENCE} \rangle \Rightarrow^* \text{the boy touches the girl with the flower}$

Example: Grammar for Regular Expressions

Example

Suppose $\Sigma = \{a, b\}$.

Consider $G_{RE_\Sigma} = (\{S\}, \Sigma \cup \{\emptyset, \epsilon, \cup, \cdot, *, (,)\}, R, S)$ where R is the set of rules:

$$S \rightarrow \emptyset \mid \epsilon \mid a \mid b \mid S \cup S \mid S \cdot S \mid S^* \mid (S)$$

$(a \cup b \cdot a)^*$ is a regular expression because
 S

Example: Grammar for Regular Expressions

Example

Suppose $\Sigma = \{a, b\}$.

Consider $G_{RE_\Sigma} = (\{S\}, \Sigma \cup \{\emptyset, \epsilon, \cup, \cdot, *, (,)\}, R, S)$ where R is the set of rules:

$$S \rightarrow \emptyset \mid \epsilon \mid a \mid b \mid S \cup S \mid S \cdot S \mid S^* \mid (S)$$

$(a \cup b \cdot a)^*$ is a regular expression because

$$S \Rightarrow_G S^*$$

Example: Grammar for Regular Expressions

Example

Suppose $\Sigma = \{a, b\}$.

Consider $G_{RE_\Sigma} = (\{S\}, \Sigma \cup \{\emptyset, \epsilon, \cup, \cdot, *, (,)\}, R, S)$ where R is the set of rules:

$$S \rightarrow \emptyset \mid \epsilon \mid a \mid b \mid S \cup S \mid S \cdot S \mid S^* \mid (S)$$

$(a \cup b \cdot a)^*$ is a regular expression because

$$S \Rightarrow_G S^* \Rightarrow_G (S)^*$$

Example: Grammar for Regular Expressions

Example

Suppose $\Sigma = \{a, b\}$.

Consider $G_{RE_\Sigma} = (\{S\}, \Sigma \cup \{\emptyset, \epsilon, \cup, \cdot, *, (,)\}, R, S)$ where R is the set of rules:

$$S \rightarrow \emptyset \mid \epsilon \mid a \mid b \mid S \cup S \mid S \cdot S \mid S^* \mid (S)$$

$(a \cup b \cdot a)^*$ is a regular expression because

$$S \Rightarrow_G S^* \Rightarrow_G (S)^* \Rightarrow_G (S \cup S)^*$$

Example: Grammar for Regular Expressions

Example

Suppose $\Sigma = \{a, b\}$.

Consider $G_{RE_\Sigma} = (\{S\}, \Sigma \cup \{\emptyset, \epsilon, \cup, \cdot, *, (,)\}, R, S)$ where R is the set of rules:

$$S \rightarrow \emptyset \mid \epsilon \mid a \mid b \mid S \cup S \mid S \cdot S \mid S^* \mid (S)$$

$(a \cup b \cdot a)^*$ is a regular expression because

$$S \Rightarrow_G S^* \Rightarrow_G (S)^* \Rightarrow_G (S \cup S)^* \Rightarrow_G (S \cup S \cdot S)^*$$

Example: Grammar for Regular Expressions

Example

Suppose $\Sigma = \{a, b\}$.

Consider $G_{RE_\Sigma} = (\{S\}, \Sigma \cup \{\emptyset, \epsilon, \cup, \cdot, *, (,)\}, R, S)$ where R is the set of rules:

$$S \rightarrow \emptyset \mid \epsilon \mid a \mid b \mid S \cup S \mid S \cdot S \mid S^* \mid (S)$$

$(a \cup b \cdot a)^*$ is a regular expression because

$$S \Rightarrow_G S^* \Rightarrow_G (S)^* \Rightarrow_G (S \cup S)^* \Rightarrow_G (S \cup S \cdot S)^* \Rightarrow_G (a \cup S \cdot S)^*$$

Example: Grammar for Regular Expressions

Example

Suppose $\Sigma = \{a, b\}$.

Consider $G_{RE_\Sigma} = (\{S\}, \Sigma \cup \{\emptyset, \epsilon, \cup, \cdot, *, (,)\}, R, S)$ where R is the set of rules:

$$S \rightarrow \emptyset \mid \epsilon \mid a \mid b \mid S \cup S \mid S \cdot S \mid S^* \mid (S)$$

$(a \cup b \cdot a)^*$ is a regular expression because

$$\begin{aligned} S &\Rightarrow_G S^* \Rightarrow_G (S)^* \Rightarrow_G (S \cup S)^* \Rightarrow_G (S \cup S \cdot S)^* \Rightarrow_G \\ &(a \cup S \cdot S)^* \Rightarrow_G (a \cup S \cdot a)^* \end{aligned}$$

Example: Grammar for Regular Expressions

Example

Suppose $\Sigma = \{a, b\}$.

Consider $G_{RE_\Sigma} = (\{S\}, \Sigma \cup \{\emptyset, \epsilon, \cup, \cdot, *, (,)\}, R, S)$ where R is the set of rules:

$$S \rightarrow \emptyset \mid \epsilon \mid a \mid b \mid S \cup S \mid S \cdot S \mid S^* \mid (S)$$

$(a \cup b \cdot a)^*$ is a regular expression because

$$\begin{aligned} S &\Rightarrow_G S^* \Rightarrow_G (S)^* \Rightarrow_G (S \cup S)^* \Rightarrow_G (S \cup S \cdot S)^* \Rightarrow_G \\ &(a \cup S \cdot S)^* \Rightarrow_G (a \cup S \cdot a)^* \Rightarrow_G (a \cup b \cdot a)^*. \end{aligned}$$

Outline

Context-Free Grammars

Parse Trees

Closure Properties of CFLs

Pushdown Automata

Equivalence of CFGs and PDAs

Grammars other than CFGs

End Matter

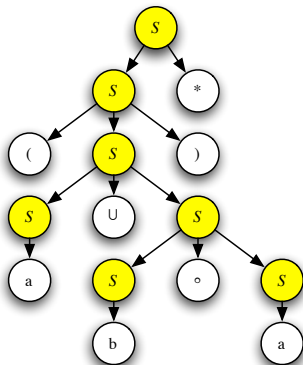
Parse Trees

A *parse tree* is a tree that shows how to derive a string from a non-terminal.

The children of a node in the tree correspond to the body of a production that has the node as head.

For $A \rightarrow \epsilon$, there is a single child, ϵ .

Parse tree for $(a \cup b \cdot a)^*$:



Yield of a Parse Tree

The concatenation of the symbols at the leaves of a parse tree is called the *yield* of the parse tree.

The yield can always be derived from the symbol at the root of the tree. If the root is S and the yield is $x \in \Sigma^*$, then $x \in L(G)$.

Leftmost Derivations

There are many ways to extract a derivation from a parse tree. If we put a restriction on how the derivation is done, we can get the derivation uniquely.

Definition

A derivation of a string w in a grammar G is a *leftmost derivation* if at every step the leftmost remaining variable is the one replaced.

Example

S

Leftmost Derivations

There are many ways to extract a derivation from a parse tree. If we put a restriction on how the derivation is done, we can get the derivation uniquely.

Definition

A derivation of a string w in a grammar G is a *leftmost derivation* if at every step the leftmost remaining variable is the one replaced.

Example

$$\underline{S} \Rightarrow_G \underline{S}^*$$

Leftmost Derivations

There are many ways to extract a derivation from a parse tree. If we put a restriction on how the derivation is done, we can get the derivation uniquely.

Definition

A derivation of a string w in a grammar G is a *leftmost derivation* if at every step the leftmost remaining variable is the one replaced.

Example

$$\underline{S} \Rightarrow_G \underline{S}^* \Rightarrow_G (\underline{S})^*$$

Leftmost Derivations

There are many ways to extract a derivation from a parse tree. If we put a restriction on how the derivation is done, we can get the derivation uniquely.

Definition

A derivation of a string w in a grammar G is a *leftmost derivation* if at every step the leftmost remaining variable is the one replaced.

Example

$$\underline{S} \Rightarrow_G \underline{S}^* \Rightarrow_G (\underline{S})^* \Rightarrow_G (\underline{S} \cup S)^*$$

Leftmost Derivations

There are many ways to extract a derivation from a parse tree. If we put a restriction on how the derivation is done, we can get the derivation uniquely.

Definition

A derivation of a string w in a grammar G is a *leftmost derivation* if at every step the leftmost remaining variable is the one replaced.

Example

$$\underline{S} \Rightarrow_G \underline{S}^* \Rightarrow_G (\underline{S})^* \Rightarrow_G (\underline{S} \cup S)^* \Rightarrow_G (a \cup \underline{S})^*$$

Leftmost Derivations

There are many ways to extract a derivation from a parse tree.
If we put a restriction on how the derivation is done, we can get the derivation uniquely.

Definition

A derivation of a string w in a grammar G is a *leftmost derivation* if at every step the leftmost remaining variable is the one replaced.

Example

$$\underline{S} \Rightarrow_G \underline{S}^* \Rightarrow_G (\underline{S})^* \Rightarrow_G (\underline{S} \cup S)^* \Rightarrow_G (a \cup \underline{S})^* \Rightarrow_G \\ (a \cup \underline{S} \cdot S)^*$$

Leftmost Derivations

There are many ways to extract a derivation from a parse tree.
If we put a restriction on how the derivation is done, we can get the derivation uniquely.

Definition

A derivation of a string w in a grammar G is a *leftmost derivation* if at every step the leftmost remaining variable is the one replaced.

Example

$$\begin{aligned} \underline{S} &\Rightarrow_G \underline{S}^* \Rightarrow_G (\underline{S})^* \Rightarrow_G (\underline{S} \cup S)^* \Rightarrow_G (a \cup \underline{S})^* \Rightarrow_G \\ (a \cup \underline{S} \cdot S)^* &\Rightarrow_G (a \cup b \cdot \underline{S})^* \end{aligned}$$

Leftmost Derivations

There are many ways to extract a derivation from a parse tree. If we put a restriction on how the derivation is done, we can get the derivation uniquely.

Definition

A derivation of a string w in a grammar G is a *leftmost derivation* if at every step the leftmost remaining variable is the one replaced.

Example

$$\underline{S} \Rightarrow_G \underline{S}^* \Rightarrow_G (\underline{S})^* \Rightarrow_G (\underline{S} \cup S)^* \Rightarrow_G (a \cup \underline{S})^* \Rightarrow_G$$
$$(a \cup \underline{S} \cdot S)^* \Rightarrow_G (a \cup b \cdot \underline{S})^* \Rightarrow_G (a \cup b \cdot a)^*$$

where we have underlined the leftmost variable at each step

Ambiguity

A CFG is *ambiguous* if there is more than one leftmost derivation for the same string.

Equivalently: more than one parse tree for the same string.

Ambiguity often causes problems:

- With interpretation.
- With parsing.

Is our grammar for regular expressions ambiguous?

Ambiguity

A CFG is *ambiguous* if there is more than one leftmost derivation for the same string.

Equivalently: more than one parse tree for the same string.

Ambiguity often causes problems:

- With interpretation.
- With parsing.

Is our grammar for regular expressions ambiguous?

Yes, but it needn't be.

Example: Unambiguous Grammar for RE_{Σ}

$G_{RE_{\Sigma}} = (\{U, C, K, T\}, \Sigma \cup \{\epsilon, \emptyset, \cup, \cdot, *, (,)\}, P, U)$ where the rules P are:

$$U \rightarrow U \cup C \mid C$$

$$C \rightarrow C \cdot K \mid K$$

$$K \rightarrow T^* \mid T$$

$$T \rightarrow (U) \mid \emptyset \mid \epsilon \mid a \mid b \mid \dots$$

$(a \cup b \cdot a)^*$ is a regular expression according to $G_{RE_{\{a,b\}}}$ because

U

Example: Unambiguous Grammar for RE_{Σ}

$G_{RE_{\Sigma}} = (\{U, C, K, T\}, \Sigma \cup \{\epsilon, \emptyset, \cup, \cdot, *, (,)\}, P, U)$ where the rules P are:

$$U \rightarrow U \cup C \mid C$$

$$C \rightarrow C \cdot K \mid K$$

$$K \rightarrow T^* \mid T$$

$$T \rightarrow (U) \mid \emptyset \mid \epsilon \mid a \mid b \mid \dots$$

$(a \cup b \cdot a)^*$ is a regular expression according to $G_{RE_{\{a,b\}}}$ because

$$U \Rightarrow_G C$$

Example: Unambiguous Grammar for RE_{Σ}

$G_{RE_{\Sigma}} = (\{U, C, K, T\}, \Sigma \cup \{\epsilon, \emptyset, \cup, \cdot, *, (,)\}, P, U)$ where the rules P are:

$$U \rightarrow U \cup C \mid C$$

$$C \rightarrow C \cdot K \mid K$$

$$K \rightarrow T^* \mid T$$

$$T \rightarrow (U) \mid \emptyset \mid \epsilon \mid a \mid b \mid \dots$$

$(a \cup b \cdot a)^*$ is a regular expression according to $G_{RE_{\{a,b\}}}$ because

$$U \Rightarrow_G C \Rightarrow_G K$$

Example: Unambiguous Grammar for RE_{Σ}

$G_{RE_{\Sigma}} = (\{U, C, K, T\}, \Sigma \cup \{\epsilon, \emptyset, \cup, \cdot, *, (,)\}, P, U)$ where the rules P are:

$$U \rightarrow U \cup C \mid C$$

$$C \rightarrow C \cdot K \mid K$$

$$K \rightarrow T^* \mid T$$

$$T \rightarrow (U) \mid \emptyset \mid \epsilon \mid a \mid b \mid \dots$$

$(a \cup b \cdot a)^*$ is a regular expression according to $G_{RE_{\{a,b\}}}$ because

$$U \Rightarrow_G C \Rightarrow_G K \Rightarrow_G T^*$$

Example: Unambiguous Grammar for RE_{Σ}

$G_{RE_{\Sigma}} = (\{U, C, K, T\}, \Sigma \cup \{\epsilon, \emptyset, \cup, \cdot, *, (,)\}, P, U)$ where the rules P are:

$$U \rightarrow U \cup C \mid C$$

$$C \rightarrow C \cdot K \mid K$$

$$K \rightarrow T^* \mid T$$

$$T \rightarrow (U) \mid \emptyset \mid \epsilon \mid a \mid b \mid \dots$$

$(a \cup b \cdot a)^*$ is a regular expression according to $G_{RE_{\{a,b\}}}$ because

$$U \Rightarrow_G C \Rightarrow_G K \Rightarrow_G T^* \Rightarrow_G (U)^*$$

Example: Unambiguous Grammar for RE_{Σ}

$G_{RE_{\Sigma}} = (\{U, C, K, T\}, \Sigma \cup \{\epsilon, \emptyset, \cup, \cdot, *, (,)\}, P, U)$ where the rules P are:

$$U \rightarrow U \cup C \mid C$$

$$C \rightarrow C \cdot K \mid K$$

$$K \rightarrow T^* \mid T$$

$$T \rightarrow (U) \mid \emptyset \mid \epsilon \mid a \mid b \mid \dots$$

$(a \cup b \cdot a)^*$ is a regular expression according to $G_{RE_{\{a,b\}}}$ because

$$U \Rightarrow_G C \Rightarrow_G K \Rightarrow_G T^* \Rightarrow_G (U)^* \Rightarrow_G (U \cup C)^*$$

Example: Unambiguous Grammar for RE_{Σ}

$G_{RE_{\Sigma}} = (\{U, C, K, T\}, \Sigma \cup \{\epsilon, \emptyset, \cup, \cdot, *, (,)\}, P, U)$ where the rules P are:

$$U \rightarrow U \cup C \mid C$$

$$C \rightarrow C \cdot K \mid K$$

$$K \rightarrow T^* \mid T$$

$$T \rightarrow (U) \mid \emptyset \mid \epsilon \mid a \mid b \mid \dots$$

$(a \cup b \cdot a)^*$ is a regular expression according to $G_{RE_{\{a,b\}}}$ because

$$U \Rightarrow_G C \Rightarrow_G K \Rightarrow_G T^* \Rightarrow_G (U)^* \Rightarrow_G (U \cup C)^* \Rightarrow_G (C \cup C)^*$$

Example: Unambiguous Grammar for RE_{Σ}

$G_{RE_{\Sigma}} = (\{U, C, K, T\}, \Sigma \cup \{\epsilon, \emptyset, \cup, \cdot, *, (,)\}, P, U)$ where the rules P are:

$$U \rightarrow U \cup C \mid C$$

$$C \rightarrow C \cdot K \mid K$$

$$K \rightarrow T^* \mid T$$

$$T \rightarrow (U) \mid \emptyset \mid \epsilon \mid a \mid b \mid \dots$$

$(a \cup b \cdot a)^*$ is a regular expression according to $G_{RE_{\{a,b\}}}$ because

$$\begin{aligned} U &\Rightarrow_G C \Rightarrow_G K \Rightarrow_G T^* \Rightarrow_G (U)^* \Rightarrow_G (U \cup C)^* \Rightarrow_G (C \cup C)^* \\ &\Rightarrow_G (K \cup C)^* \end{aligned}$$

Example: Unambiguous Grammar for RE_Σ

$G_{\text{RE}_\Sigma} = (\{U, C, K, T\}, \Sigma \cup \{\epsilon, \emptyset, \cup, \cdot, *, (,)\}, P, U)$ where the rules P are:

$$U \rightarrow U \cup C \mid C$$

$$C \rightarrow C \cdot K \mid K$$

$$K \rightarrow T^* \mid T$$

$$T \rightarrow (U) \mid \emptyset \mid \epsilon \mid a \mid b \mid \dots$$

$(a \cup b \cdot a)^*$ is a regular expression according to $G_{\text{RE}_{\{a,b\}}}$ because

$$\begin{aligned} U &\Rightarrow_G C \Rightarrow_G K \Rightarrow_G T^* \Rightarrow_G (U)^* \Rightarrow_G (U \cup C)^* \Rightarrow_G (C \cup C)^* \\ &\Rightarrow_G (K \cup C)^* \Rightarrow_G (T \cup C)^* \end{aligned}$$

Example: Unambiguous Grammar for RE_{Σ}

$G_{RE_{\Sigma}} = (\{U, C, K, T\}, \Sigma \cup \{\epsilon, \emptyset, \cup, \cdot, *, (,)\}, P, U)$ where the rules P are:

$$U \rightarrow U \cup C \mid C$$

$$C \rightarrow C \cdot K \mid K$$

$$K \rightarrow T^* \mid T$$

$$T \rightarrow (U) \mid \emptyset \mid \epsilon \mid a \mid b \mid \dots$$

$(a \cup b \cdot a)^*$ is a regular expression according to $G_{RE_{\{a,b\}}}$ because

$$\begin{aligned} U &\Rightarrow_G C \Rightarrow_G K \Rightarrow_G T^* \Rightarrow_G (U)^* \Rightarrow_G (U \cup C)^* \Rightarrow_G (C \cup C)^* \\ &\Rightarrow_G (K \cup C)^* \Rightarrow_G (T \cup C)^* \Rightarrow_G (a \cup C)^* \end{aligned}$$

Example: Unambiguous Grammar for RE_{Σ}

$G_{RE_{\Sigma}} = (\{U, C, K, T\}, \Sigma \cup \{\epsilon, \emptyset, \cup, \cdot, *, (,)\}, P, U)$ where the rules P are:

$$U \rightarrow U \cup C \mid C$$

$$C \rightarrow C \cdot K \mid K$$

$$K \rightarrow T^* \mid T$$

$$T \rightarrow (U) \mid \emptyset \mid \epsilon \mid a \mid b \mid \dots$$

$(a \cup b \cdot a)^*$ is a regular expression according to $G_{RE_{\{a,b\}}}$ because

$$\begin{aligned} U &\Rightarrow_G C \Rightarrow_G K \Rightarrow_G T^* \Rightarrow_G (U)^* \Rightarrow_G (U \cup C)^* \Rightarrow_G (C \cup C)^* \\ &\Rightarrow_G (K \cup C)^* \Rightarrow_G (T \cup C)^* \Rightarrow_G (a \cup C)^* \Rightarrow_G (a \cup C \cdot K)^* \end{aligned}$$

Example: Unambiguous Grammar for RE_{Σ}

$G_{RE_{\Sigma}} = (\{U, C, K, T\}, \Sigma \cup \{\epsilon, \emptyset, \cup, \cdot, *, (,)\}, P, U)$ where the rules P are:

$$U \rightarrow U \cup C \mid C$$

$$C \rightarrow C \cdot K \mid K$$

$$K \rightarrow T^* \mid T$$

$$T \rightarrow (U) \mid \emptyset \mid \epsilon \mid a \mid b \mid \dots$$

$(a \cup b \cdot a)^*$ is a regular expression according to $G_{RE_{\{a,b\}}}$ because

$$\begin{aligned} U &\Rightarrow_G C \Rightarrow_G K \Rightarrow_G T^* \Rightarrow_G (U)^* \Rightarrow_G (U \cup C)^* \Rightarrow_G (C \cup C)^* \\ &\Rightarrow_G (K \cup C)^* \Rightarrow_G (T \cup C)^* \Rightarrow_G (a \cup C)^* \Rightarrow_G (a \cup C \cdot K)^* \\ &\Rightarrow_G (a \cup K \cdot K)^* \end{aligned}$$

Example: Unambiguous Grammar for RE_{Σ}

$G_{RE_{\Sigma}} = (\{U, C, K, T\}, \Sigma \cup \{\epsilon, \emptyset, \cup, \cdot, *, (,)\}, P, U)$ where the rules P are:

$$U \rightarrow U \cup C \mid C$$

$$C \rightarrow C \cdot K \mid K$$

$$K \rightarrow T^* \mid T$$

$$T \rightarrow (U) \mid \emptyset \mid \epsilon \mid a \mid b \mid \dots$$

$(a \cup b \cdot a)^*$ is a regular expression according to $G_{RE_{\{a,b\}}}$ because

$$\begin{aligned} U &\Rightarrow_G C \Rightarrow_G K \Rightarrow_G T^* \Rightarrow_G (U)^* \Rightarrow_G (U \cup C)^* \Rightarrow_G (C \cup C)^* \\ &\Rightarrow_G (K \cup C)^* \Rightarrow_G (T \cup C)^* \Rightarrow_G (a \cup C)^* \Rightarrow_G (a \cup C \cdot K)^* \\ &\Rightarrow_G (a \cup K \cdot K)^* \Rightarrow_G (a \cup T \cdot K)^* \end{aligned}$$

Example: Unambiguous Grammar for RE_{Σ}

$G_{RE_{\Sigma}} = (\{U, C, K, T\}, \Sigma \cup \{\epsilon, \emptyset, \cup, \cdot, *, (,)\}, P, U)$ where the rules P are:

$$U \rightarrow U \cup C \mid C$$

$$C \rightarrow C \cdot K \mid K$$

$$K \rightarrow T^* \mid T$$

$$T \rightarrow (U) \mid \emptyset \mid \epsilon \mid a \mid b \mid \dots$$

$(a \cup b \cdot a)^*$ is a regular expression according to $G_{RE_{\{a,b\}}}$ because

$$\begin{aligned} U &\Rightarrow_G C \Rightarrow_G K \Rightarrow_G T^* \Rightarrow_G (U)^* \Rightarrow_G (U \cup C)^* \Rightarrow_G (C \cup C)^* \\ &\Rightarrow_G (K \cup C)^* \Rightarrow_G (T \cup C)^* \Rightarrow_G (a \cup C)^* \Rightarrow_G (a \cup C \cdot K)^* \\ &\Rightarrow_G (a \cup K \cdot K)^* \Rightarrow_G (a \cup T \cdot K)^* \Rightarrow_G (a \cup b \cdot K)^* \end{aligned}$$

Example: Unambiguous Grammar for RE_{Σ}

$G_{RE_{\Sigma}} = (\{U, C, K, T\}, \Sigma \cup \{\epsilon, \emptyset, \cup, \cdot, *, (,)\}, P, U)$ where the rules P are:

$$U \rightarrow U \cup C \mid C$$

$$C \rightarrow C \cdot K \mid K$$

$$K \rightarrow T^* \mid T$$

$$T \rightarrow (U) \mid \emptyset \mid \epsilon \mid a \mid b \mid \dots$$

$(a \cup b \cdot a)^*$ is a regular expression according to $G_{RE_{\{a,b\}}}$ because

$$\begin{aligned} U &\Rightarrow_G C \Rightarrow_G K \Rightarrow_G T^* \Rightarrow_G (U)^* \Rightarrow_G (U \cup C)^* \Rightarrow_G (C \cup C)^* \\ &\Rightarrow_G (K \cup C)^* \Rightarrow_G (T \cup C)^* \Rightarrow_G (a \cup C)^* \Rightarrow_G (a \cup C \cdot K)^* \\ &\Rightarrow_G (a \cup K \cdot K)^* \Rightarrow_G (a \cup T \cdot K)^* \Rightarrow_G (a \cup b \cdot K)^* \\ &\Rightarrow_G (a \cup b \cdot T)^* \end{aligned}$$

Example: Unambiguous Grammar for RE_{Σ}

$G_{RE_{\Sigma}} = (\{U, C, K, T\}, \Sigma \cup \{\epsilon, \emptyset, \cup, \cdot, *, (,)\}, P, U)$ where the rules P are:

$$U \rightarrow U \cup C \mid C$$

$$C \rightarrow C \cdot K \mid K$$

$$K \rightarrow T^* \mid T$$

$$T \rightarrow (U) \mid \emptyset \mid \epsilon \mid a \mid b \mid \dots$$

$(a \cup b \cdot a)^*$ is a regular expression according to $G_{RE_{\{a,b\}}}$ because

$$\begin{aligned} U &\Rightarrow_G C \Rightarrow_G K \Rightarrow_G T^* \Rightarrow_G (U)^* \Rightarrow_G (U \cup C)^* \Rightarrow_G (C \cup C)^* \\ &\Rightarrow_G (K \cup C)^* \Rightarrow_G (T \cup C)^* \Rightarrow_G (a \cup C)^* \Rightarrow_G (a \cup C \cdot K)^* \\ &\Rightarrow_G (a \cup K \cdot K)^* \Rightarrow_G (a \cup T \cdot K)^* \Rightarrow_G (a \cup b \cdot K)^* \\ &\Rightarrow_G (a \cup b \cdot T)^* \Rightarrow_G (a \cup b \cdot a)^*. \end{aligned}$$

Inherently Ambiguous CFLs

Some languages are context-free but don't have unambiguous CFGs.

$$\left\{ a^i b^j c^k \mid i = j \vee j = k \right\}$$

Intuition: This is the union of two unambiguous grammars, but they have to overlap when $i = j = k$.

$$S \rightarrow AC \mid BD$$

$$A \rightarrow aAb \mid \epsilon$$

$$C \rightarrow Cc \mid \epsilon$$

$$D \rightarrow bDc \mid \epsilon$$

$$B \rightarrow Ba \mid \epsilon$$

Outline

Context-Free Grammars

Parse Trees

Closure Properties of CFLs

Pushdown Automata

Equivalence of CFGs and PDAs

Grammars other than CFGs

End Matter

Closure Properties of CFLs

Theorem

CFLs are closed under union.

Proof.

Let $L_1, L_2 \subseteq \Sigma^*$ be context-free.

For $i = 1, 2$ let $G_i = (V_i, \Sigma, P_i, S_i)$ be a CFG such that $L(G_i) = L_i$ such that w.l.o.g. $V_1 \cap V_2 = \emptyset$.

Consider $G = (V_1 \cup V_2 \cup \{S\}, \Sigma, P, S)$ for some fresh non-terminal $S \notin V_1 \cup V_2 \cup \Sigma$ where

$$P = P_1 \cup P_2 \cup \{S \rightarrow S_1 \mid S_2\}$$

This grammar clearly generates $L_1 \cup L_2$. □

Closure Properties of CFLs

Theorem

CFLs are closed under concatenation.

Proof.

With L_1, L_2, G_1, G_2, G as in the previous proof, except for

$$P = P_1 \cup P_2 \cup \{S \rightarrow S_1 S_2\}$$

This grammar clearly generates $L_1 L_2$. □

Closure Properties of CFLs

Theorem

CFLs are closed under Kleene star ($$).*

Proof: exercise.

Closure Properties of CFLs

What about intersection? complement?

Outline

Context-Free Grammars

Parse Trees

Closure Properties of CFLs

Pushdown Automata

Equivalence of CFGs and PDAs

Grammars other than CFGs

End Matter

Pushdown Automata

Pushdown Automata are to CFGs what Finite Automata are to Regular Expressions.

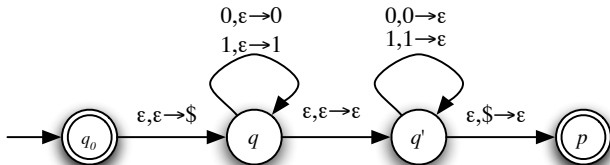
A PDA is an ϵ -NFA with an additional *stack*.

The stack makes it more powerful than an NFA because states can only “store” a fixed amount of information, while the stack is unbounded.

But the stack can only be used in a limited way, by pushing and popping symbols.

PDA

Here is an example PDA that accepts the language $\{ w \text{rev}(w) \mid w \in \Sigma^* \}$ (even-length palindromes):



In this case, the input alphabet is $\{0, 1\}$ and the stack alphabet is $\{0, 1, \$\}$.

Rule $x, y \rightarrow z$ intuitively means “read input x , replace the y at the top of the stack by z ”. It applies only if there *is* a y at the top of the stack!

PDA Formalities

Definition

A *pushdown automaton* is a 6-tuple $(Q, \Sigma, \Gamma, \delta, q_0, F)$ where Q , Σ , and Γ are all finite sets, and

- 1 Q is the set of *states*,
- 2 Σ is the *input alphabet*,
- 3 Γ is the *stack alphabet*,
- 4 $\delta : Q \times \Sigma_{\epsilon} \times \Gamma_{\epsilon} \longrightarrow 2^{Q \times \Gamma_{\epsilon}}$ is a *transition relation*,
- 5 $q_0 \in Q$ is the *start state*, and
- 6 $F \subseteq Q$ is the set of *accept states*.

Acceptance by a PDA

Definition

Let $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$ be a PDA. An *instantaneous description* (or *ID*) is a snapshot $\iota = (q, w, \alpha)$ of the PDA recording

- the current state $q \in Q$,
- the input that has not yet been read $w \in \Sigma^*$, and
- the complete contents of the stack $\alpha \in \Gamma^*$.

An ID has everything necessary to predict the possible future IDs of the PDA.

Acceptance

IDs evolve over time. $\iota_i \rightsquigarrow \iota_{i+1}$ if the PDA can transform ι_i to ι_{i+1} :

Definition

Define $\rightsquigarrow$ by

$$(q, aw, X\beta) \rightsquigarrow (p, w, \alpha\beta)$$

if $\delta(q, a, X)$ contains (p, α) .

Define $\rightsquigarrow^*$ to be the reflexive transitive closure of $\rightsquigarrow$.

Definition

A string w is accepted if there exists a γ such that $(q_0, w, \epsilon) \rightsquigarrow^* (p, \epsilon, \gamma)$ and $p \in F$.

$$L(P) = \{ w \in \Sigma^* \mid \exists p \in F, \gamma \in \Gamma^* ((q_0, w, \epsilon) \rightsquigarrow^* (p, \epsilon, \gamma)) \}$$

PDA Acceptance Example

Running the previous PDA on input 0110 gives the following computation

$(q_0, 0110, \epsilon)$

PDA Acceptance Example

Running the previous PDA on input 0110 gives the following computation

$$(q_0, 0110, \epsilon) \rightsquigarrow (q, 0110, \$)$$

PDA Acceptance Example

Running the previous PDA on input 0110 gives the following computation

$$(q_0, 0110, \epsilon) \rightsquigarrow (q, 0110, \$) \rightsquigarrow (q, 110, 0\$)$$

PDA Acceptance Example

Running the previous PDA on input 0110 gives the following computation

$$(q_0, 0110, \epsilon) \rightsquigarrow (q, 0110, \$) \rightsquigarrow (q, 110, 0\$) \rightsquigarrow (q, 10, 10\$)$$

PDA Acceptance Example

Running the previous PDA on input 0110 gives the following computation

$$(q_0, 0110, \epsilon) \rightsquigarrow (q, 0110, \$) \rightsquigarrow (q, 110, 0\$) \rightsquigarrow (q, 10, 10\$) \rightsquigarrow (q', 10, 10\$)$$

PDA Acceptance Example

Running the previous PDA on input 0110 gives the following computation

$$(q_0, 0110, \epsilon) \rightsquigarrow (q, 0110, \$) \rightsquigarrow (q, 110, 0\$) \rightsquigarrow (q, 10, 10\$) \rightsquigarrow (q', 10, 10\$) \rightsquigarrow (q', 0, 0\$)$$

PDA Acceptance Example

Running the previous PDA on input 0110 gives the following computation

$$\begin{aligned} (q_0, 0110, \epsilon) &\rightsquigarrow (q, 0110, \$) \rightsquigarrow (q, 110, 0\$) \rightsquigarrow (q, 10, 10\$) \rightsquigarrow \\ &(q', 10, 10\$) \rightsquigarrow (q', 0, 0\$) \rightsquigarrow (q', \epsilon, \$) \end{aligned}$$

PDA Acceptance Example

Running the previous PDA on input 0110 gives the following computation

$$\begin{aligned} (q_0, 0110, \epsilon) &\rightsquigarrow (q, 0110, \$) \rightsquigarrow (q, 110, 0\$) \rightsquigarrow (q, 10, 10\$) \rightsquigarrow \\ &(q', 10, 10\$) \rightsquigarrow (q', 0, 0\$) \rightsquigarrow (q', \epsilon, \$) \rightsquigarrow (p, \epsilon, \epsilon) \end{aligned}$$

Outline

Context-Free Grammars

Parse Trees

Closure Properties of CFLs

Pushdown Automata

Equivalence of CFGs and PDAs

Grammars other than CFGs

End Matter

Theorem

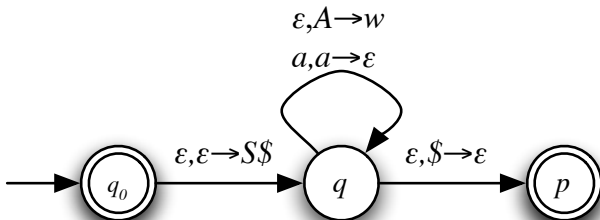
$L \subseteq \Sigma^*$ is context-free iff some PDA recognises L .

CFG $\longrightarrow$ PDA

Proof idea: Let the PDA guess a derivation, then match what it derived against the input word.

Refinement of the idea: Let the PDA guess a derivation while matching terminals from the input word.

CFG $\longrightarrow$ PDA



for every rule $A \rightarrow w$ and terminal a . Transitions $x, y \rightarrow z$ where z is a word (rather than just a single symbol) are abbreviations for detours through auxiliary states to push z onto the stack one symbol at a time (in reverse, so the left of z is at the top).

The details are in [Sipser2006].

Example: Consider the CFG for alphabet $\{0, 1\}$ with start symbol S and productions $S \rightarrow 0S1$ and $S \rightarrow \epsilon$. (This accepts $L = \{0^i 1^i \mid i \in \mathbb{N}\}$)

The following is a computation of the corresponding PDA on input 0011:

$$\begin{aligned} (q_0, 0011, \epsilon) &\rightsquigarrow^* (q, 0011, S\$) \rightsquigarrow^* (q, 0011, 0S1\$) \rightsquigarrow \\ (q, 011, S1\$) &\rightsquigarrow^* (q, 011, 0S11\$) \rightsquigarrow (q, 11, S11\$) \rightsquigarrow \\ (q, 11, 11\$) &\rightsquigarrow (q, 1, 1\$) \rightsquigarrow (q, \epsilon, \$) \rightsquigarrow (p, \epsilon, \epsilon) \end{aligned}$$

PDA $\longrightarrow$ CFG

Start with a PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$.

Design a CFG $G = (V, \Sigma, R, S)$ with a non-terminal A_{pq} for each pair $(p, q) \in Q \times Q$ with the idea that $A_{p,q}$ generates all those strings that take P from state p with an empty stack to state q with an empty stack:

$$V = \{ A_{pq} \mid p, q \in Q \}$$

$$S = A_{q_0 q_{\text{accept}}}$$

(This also means that strings generated by A_{pq} take P from state p with stack content γ to state q with stack content γ without ever dipping into γ .)

PDA $\longrightarrow$ CFG, cont.

First we transform the machine into one that

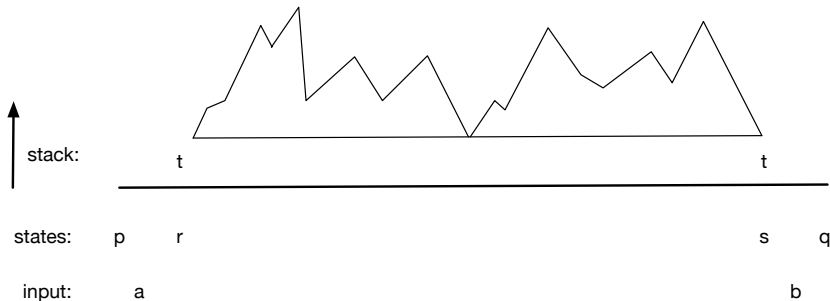
- 1 has a single accept state $F = \{q_{\text{accept}}\}$
- 2 empties its stack before terminating
- 3 has only transitions that either push or pop a symbol from the stack (but not both in a single transition, nor transitions that neither push nor pop.)

PDA $\longrightarrow$ CFG, cont.

R consists of the following rules:

$$A_{pq} \rightarrow aA_{rs}b \text{ if } p \xrightarrow{a, \epsilon \rightarrow t} r \text{ and } s \xrightarrow{b, t \rightarrow \epsilon} q$$

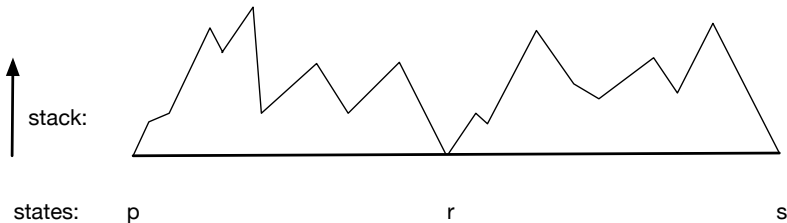
for some $p, q, r, s \in Q$, $t \in \Gamma$ and $a, b \in \Sigma_\epsilon$



PDA $\longrightarrow$ CFG, cont.

R consists of the following rules:

$$A_{pq} \rightarrow A_{pr}A_{rq}, \text{ for } p, q, r \in Q$$



$$A_{pp} \rightarrow \epsilon, \text{ for } p \in Q$$

Lemma

If $A_{pq} \Rightarrow_G^ x \in \Sigma^*$ then x can take P from state p with an empty stack to state q with an empty stack.*

Lemma

If $x \in \Sigma^$ can take P from state p with an empty stack to state q with an empty stack then $A_{pq} \Rightarrow_G^* x$.*

The lemmas are proven in [Sipser2006].

Outline

Context-Free Grammars

Parse Trees

Closure Properties of CFLs

Pushdown Automata

Equivalence of CFGs and PDAs

Grammars other than CFGs

End Matter

Grammars for Regular Languages

Clearly an NFA is also a PDA, so every regular language is a CFL.

$$\text{REGULAR} \subseteq \text{CONTEXT-FREE}$$

Question

Is there a grammar for regular languages?

Right-Linear Grammars

Definition

A CFG (V, Σ, P, S) is **right-linear** if all productions in P are of the form $A \rightarrow w$ or $A \rightarrow wB$, where $w \in \Sigma$ and $B \in V$.

Right-Linear Grammars

Definition

A CFG (V, Σ, P, S) is **right-linear** if all productions in P are of the form $A \rightarrow w$ or $A \rightarrow wB$, where $w \in \Sigma$ and $B \in V$.

Theorem

A language L is regular if and only if it can be generated by a right-linear grammar.

Right-Linear Grammars

Definition

A CFG (V, Σ, P, S) is **right-linear** if all productions in P are of the form $A \rightarrow w$ or $A \rightarrow wB$, where $w \in \Sigma$ and $B \in V$.

Theorem

A language L is regular if and only if it can be generated by a right-linear grammar.

Sketch of proof.

Use the grammar to define an NFA with a unique accept state q_f (and conversely):

- Non-final states correspond to variables
- Transitions given by rules:
 - e.g. $A \rightarrow wB$ indicates that $B \in \delta(A, w)$
 - e.g. $A \rightarrow w$ indicates that $q_f \in \delta(A, w)$



Grammars

CFGs fall into a larger class of grammars, the Chomsky grammars

Definition (Chomsky Grammar)

A *Chomsky Grammar* is a 4-tuple $G = (N, \Sigma, P, S)$ where N , Σ , and S are as for CFGs, but the finite set P of *productions* satisfies

$$P \subseteq ((N \cup \Sigma)^* N (N \cup \Sigma)^*) \times (N \cup \Sigma)^* .$$

That is, each production in P is a rewrite rule

$$\alpha \rightarrow \beta$$

where $\alpha, \beta \in (N \cup \Sigma)^*$ and α contains at least one nonterminal (element of N)

Definition (Chomsky Hierarchy)

A grammar $G = (N, \Sigma, P, S)$ is of type

- 0 (or *recursively enumerable*) in the general case.
- 1 (or *context-sensitive*), if all productions are of the form $\alpha A \beta \rightarrow \alpha \gamma \beta$, where A is a nonterminal and $\gamma \neq \epsilon$, except that we allow $S \rightarrow \epsilon$, provided there is no S on the RHS of any rule.
- 2 (or *context-free*), if all productions have the form $A \rightarrow \alpha$.
- 3 (or *right-linear*), if all productions are of the form $A \rightarrow w$ or $A \rightarrow wB$, where $w \in \Sigma$ and $B \in N$.

The language $L \subseteq \Sigma^*$ is said to be of type i if there exists a grammar G of the respective type with $L = L(G)$.

Not-so-serious question

Where does natural language sit?

English is not regular

The mouse ate the cheese.

English is not regular

The mouse ate the cheese.

The mouse, the cat chased, ate the cheese.

English is not regular

The mouse ate the cheese.

The mouse, the cat chased, ate the cheese.

The mouse, the cat, the dog watched, chased, ate the cheese.

etc.

Outline

Context-Free Grammars

Parse Trees

Closure Properties of CFLs

Pushdown Automata

Equivalence of CFGs and PDAs

Grammars other than CFGs

End Matter

Weekly Feedback

I would appreciate any comments/suggestions/requests you have on this week's lectures (and tutorials).



<https://forms.office.com/r/g3LgPASave>